

Resource dependencies (depends_on)

◆ What Are Resource Dependencies in Terraform?

In Terraform, resource dependencies determine the order in which resources are created, updated, and destroyed.

Terraform automatically builds a dependency graph based on how resources reference each other. However, you can manually declare dependencies using `depends_on` when Terraform can't detect them implicitly.

◆ How Terraform Builds Dependencies *Automatically*

When refer one resource inside another (like using its attribute), Terraform automatically understands that one must be created before the other

✓ Example – Automatic Dependency

```
resource "aws_vpc" "main" {  
  cidr_block = "10.0.0.0/16"  
}
```

```
resource "aws_subnet" "subnet1" {  
  vpc_id      = aws_vpc.main.id # <- Implicit dependency  
  cidr_block = "10.0.1.0/24"  
}
```

Here, Terraform sees `aws_subnet.subnet1` uses `aws_vpc.main.id`, so it **knows** it must create the VPC first, then the subnet.

◆ When Do You Need **depends_on**?

We use **depends_on** when:

1. No direct attribute references exist between resources.
2. Terraform cannot automatically detect that one resource depends on another.
3. You want to enforce an order due to external dependencies (like in modules or null resources).
4. Working with side effects (like IAM roles, policies, scripts, or provisioning).

◆ Syntax of **depends_on**

```
resource "aws_instance" "example" {  
  ami          = "ami-0c55b159cbfaffe1f0"  
  instance_type = "t2.micro"  
  
  depends_on = [aws_security_group.allow_ssh]  
}
```

Here, **aws_instance.example** **won't be created** until **aws_security_group.allow_ssh** is successfully created.

◆ **depends_on with Modules**

If using modules, and you want to ensure one module runs after another:

```
module "network" {  
    source = "../network"  
}
```

```
module "compute" {  
    source = "../compute"  
  
    depends_on = [module.network]  
}
```

This ensures `module.compute` starts **after** `module.network` completes.

◆ **depends_on in null_resource**

null_resource is often used for provisioning or running scripts using **local-exec** or **remote-exec**.

```
resource "null_resource" "example" {  
    depends_on = [aws_instance.web]  
  
    provisioner "local-exec" {  
        command = "echo 'Instance is created'"  
    }  
}
```

```
}
```

This ensures the script only runs **after** `aws_instance.web` is fully created.

◆ What Happens on Destroy?

Terraform also uses the dependency graph for destruction. Resources are destroyed in reverse order of creation, respecting `depends_on`.

- If A depends on B, then A is destroyed first, then B.

✅ What are Resource Dependencies in Terraform?

Terraform builds and destroys infrastructure in a specific order, based on how resources depend on each other.

There are two types of dependencies in Terraform:

1. Implicit Dependencies ✅ (automatic)
2. Explicit Dependencies ✅ (manual using `depends_on`)

◆ 1. Implicit Dependencies (Automatic)

An implicit dependency is when Terraform can automatically detect the relationship between resources — because one resource is using data from another.

📦 Example:

```
resource "aws_vpc" "main" {  
  cidr_block = "10.0.0.0/16"  
}
```

```
resource "aws_subnet" "subnet" {  
  vpc_id = aws_vpc.main.id # Implicit dependency
```

```
cidr_block = "10.0.1.0/24"

}
```

✓ What's happening?

- `aws_subnet.subnet` is **implicitly dependent** on `aws_vpc.main`.
- Terraform sees that `subnet` needs the `vpc_id`, so it **creates the VPC first**, then the subnet.

♦ 2. Explicit Dependencies (Manual using `depends_on`)

An **explicit dependency** is when we manually tell Terraform:

"Don't create this resource until that one is done."

We use the `depends_on` meta-argument to define this.

📦 Example:

```
resource "aws_instance" "web" {

  ami          = "ami-123456"

  instance_type = "t2.micro"

}
```

```
resource "null_resource" "after_instance" {

  depends_on = [aws_instance.web] # Explicit dependency

  provisioner "local-exec" {

    command = "echo 'EC2 is ready!'"

  }

}
```

```
}
```

✓ What's happening?

`null_resource.after_instance` doesn't reference the instance directly.

But we **force Terraform** to wait for `aws_instance.web` by using `depends_on`.